

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY
Faculty of Technology & Engineering
Devang Patel Institute of Advance Technology and Research
(DEPSTAR)

CSUE301 – : Big Data Analytics
Practical File – ODD Semester 2026-27

StudentID: 24DIT041
Student Name: KRINA PATEL

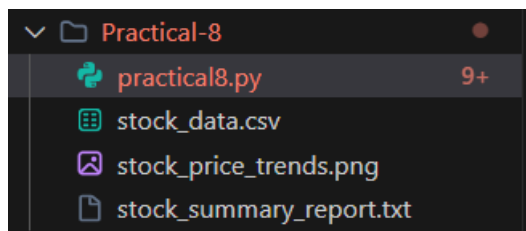
Practical 8

1. Problem Definition

You are a Data Analytics Engineer at FinEdge Securities Ltd., a financial services company that provides stock market insights to investors. The company continuously receives stock price updates from multiple exchanges. Investors want to understand stock performance trends and identify the best investment opportunities. Traditional data processing systems struggle to handle the continuously growing volume of financial data.

PART A — Practical Execution

Step 1: Creating Folder and files



Step 2: Creating Spark Session

```
=====
TASK 1: CREATING SPARK SESSION
=====
WARNING: Using incubator modules: jdk.incubator.vector
26/09/27 16:41:24 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: java.io.FileNotFoundException: HADOOP_HOME and hadoop.home.dir are unset. -see
https://wiki.apache.org/confluence/display/HADOOP2/WindowsProblems
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/27 16:41:24 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Spark Version   : 4.2.0
Spark App Name  : Stock-Market-Analytics
Master URL      : local[*]
```

Step 3: Loading Stock Dataset

```
=====
TASK 2: LOADING STOCK DATASET
=====
Dataset Path      : C:\Users\ADMIN\OneDrive\Desktop\BDA\Practical-8\stock_data.csv
Dataset loaded successfully.
Total Number of Records: 300

First 10 Records:
+-----+-----+-----+
|Timestamp|Stock|Price|
+-----+-----+-----+
|2026-01-05|AAPL|185.0|
|2026-01-05|GOOGL|160.0|
|2026-01-05|MSFT|415.0|
|2026-01-05|AMZN|175.0|
|2026-01-05|TSLA|210.0|
|2026-01-06|AAPL|185.03|
|2026-01-06|GOOGL|159.9|
|2026-01-06|MSFT|415.01|
|2026-01-06|AMZN|177.05|
|2026-01-06|TSLA|209.76|
+-----+-----+-----+
only showing top 10 rows
```

Step 4: Display Dataset Schema

```
=====
TASK 3: DISPLAYING SCHEMA AND SAMPLE RECORDS
=====
DataFrame Schema:
root
 |-- Timestamp: date (nullable = true)
 |-- Stock: string (nullable = true)
 |-- Price: double (nullable = true)
```

Step 5: Group Records by Stock Symbol

```
=====
TASK 4: GROUPING BY STOCK SYMBOL
=====
```

Stock	Number_of_Records
AAPL	60
AMZN	60
GOOGL	60
MSFT	60
TSLA	60

Step 6: Calculate Average Stock Price

```
=====
TASK 5: AVERAGE STOCK PRICE
=====
```

Stock	Average_Price
MSFT	420.1
TSLA	246.48
AAPL	202.22
AMZN	190.48
GOOGL	178.72

Step 7: Calculate Maximum Stock Price

```
=====
TASK 6: MAXIMUM STOCK PRICE
=====
```

Stock	Maximum_Price
MSFT	435.31
TSLA	341.0
AAPL	230.34
AMZN	205.17
GOOGL	191.89

Step 8: Calculate Minimum Stock Price

```
=====
TASK 7: MINIMUM STOCK PRICE
=====
+-----+-----+
|Stock|Minimum_Price|
+-----+-----+
|GOOGL|159.69      |
|AMZN |175.0       |
|AAPL |181.07      |
|TSLA |192.81      |
|MSFT |409.53      |
+-----+-----+
```

Step 9: Generate Stock Summary Report

```
=====
TASK 8: STOCK SUMMARY REPORT
=====
+-----+-----+-----+-----+-----+
|Stock|Total_Records|Average_Price|Minimum_Price|Maximum_Price|
+-----+-----+-----+-----+-----+
|AAPL |60           |202.22      |181.07      |230.34      |
|AMZN |60           |190.48      |175.0       |205.17      |
|GOOGL|60           |178.72      |159.69      |191.89      |
|MSFT |60           |420.1       |409.53      |435.31      |
|TSLA |60           |246.48      |192.81      |341.0       |
+-----+-----+-----+-----+-----+
```

Step 10: Identify Best-Performing Stock

```
=====
TASK 9: BEST-PERFORMING STOCKS
=====
+-----+-----+-----+-----+
|Stock|First_Price|Last_Price|Return_Percentage|
+-----+-----+-----+-----+
|TSLA |210.0      |331.7     |57.95           |
|AAPL |185.0      |230.34    |24.51           |
|GOOGL|160.0      |186.04    |16.27           |
|AMZN |175.0      |193.18    |10.39           |
|MSFT |415.0      |410.46    |-1.09           |
+-----+-----+-----+-----+

[>>>] Best-Performing Stock: TSLA with Return of 57.95%
```

Step 11: Calculate Daily Stock Returns

===== SUPPLEMENTARY 1: DAILY STOCK RETURNS =====

Using Window: Window.partitionBy('Stock').orderBy('Timestamp') with lag('Price')

Sample Daily Returns (First 10 records):

Timestamp	Stock	Price	Previous_Price	Daily_Return_Percentage
2026-01-05	AAPL	185.0	NULL	0.0
2026-01-06	AAPL	185.03	185.0	0.02
2026-01-07	AAPL	182.09	185.03	-1.59
2026-01-08	AAPL	182.95	182.09	0.47
2026-01-09	AAPL	181.07	182.95	-1.03
2026-01-10	AAPL	182.59	181.07	0.84
2026-01-11	AAPL	182.41	182.59	-0.1
2026-01-12	AAPL	184.01	182.41	0.88
2026-01-13	AAPL	186.29	184.01	1.24
2026-01-14	AAPL	186.45	186.29	0.09

only showing top 10 rows

Step 12: Identify Top-Performing Stock

===== SUPPLEMENTARY 2: TOP-PERFORMING STOCK =====

Ranked Stocks by Cumulative Period Return:

Stock	First_Price	Last_Price	Return_Percentage
TSLA	210.0	331.7	57.95
AAPL	185.0	230.34	24.51
GOOGL	160.0	186.04	16.27
AMZN	175.0	193.18	10.39
MSFT	415.0	410.46	-1.09

Top-Ranked Stock overall : TSLA (57.95%)

Step 13: Calculate 3-Period Moving Average

```
=====
SUPPLEMENTARY 3: MOVING AVERAGE
=====
Computing 3-Period Moving Average using rowsBetween(-2, 0)...

Sample Moving Average Results (First 10 records):
+-----+-----+-----+-----+
|Timestamp|Stock|Price|3_Day_Moving_Avg|
+-----+-----+-----+-----+
|2026-01-05|AAPL|185.0|185.0|
|2026-01-06|AAPL|185.03|185.02|
|2026-01-07|AAPL|182.09|184.04|
|2026-01-08|AAPL|182.95|183.36|
|2026-01-09|AAPL|181.07|182.04|
|2026-01-10|AAPL|182.59|182.2|
|2026-01-11|AAPL|182.41|182.02|
|2026-01-12|AAPL|184.01|183.0|
|2026-01-13|AAPL|186.29|184.24|
|2026-01-14|AAPL|186.45|185.58|
+-----+-----+-----+-----+
only showing top 10 rows
```

Step 14: Calculate Stock Volatility

```
=====
SUPPLEMENTARY 4: STOCK VOLATILITY
=====
Calculating Volatility as Standard Deviation of Daily Returns...
+-----+-----+
|Stock|Volatility_StdDev|
+-----+-----+
|TSLA|2.46|
|AMZN|1.38|
|GOOGL|1.11|
|AAPL|0.98|
|MSFT|0.8|
+-----+-----+
```

Step 15: Market Trend Analysis

```
=====
TASK 10: MARKET TREND ANALYSIS
=====
1. Stocks with Positive Cumulative Return : TSLA, AAPL, GOOGL, AMZN
2. Stocks with Negative Cumulative Return : MSFT
3. Stock with Highest Average Price      : MSFT ($420.10)
4. Highest Single Stock Price Peak      : MSFT ($435.31)
5. Lowest Single Stock Price Trough     : GOOGL ($159.69)

=====
TASK 11: INVESTMENT RECOMMENDATIONS
=====
[DISCLAIMER]: Educational analysis only -- not financial advice.

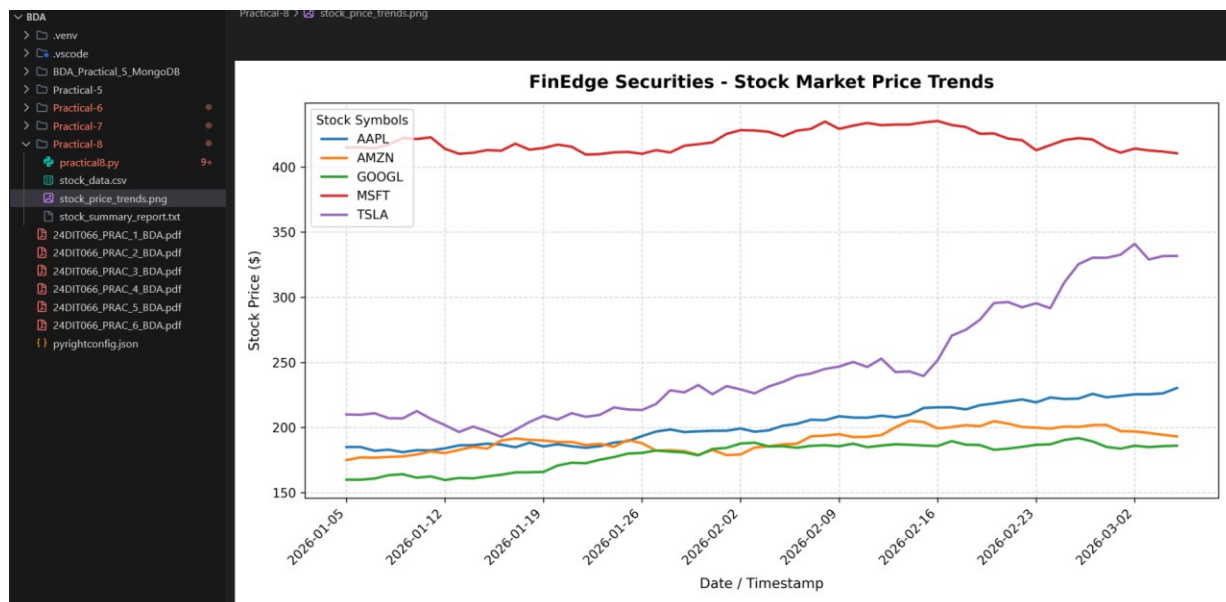
[*] TSLA : Return = 57.95% | Volatility = 2.46% -> Profile: Aggressive Growth (High Return, Elevated Volatility)
[*] AAPL : Return = 24.51% | Volatility = 0.98% -> Profile: Balanced / Moderate (Steady Positive Return, Low Volatility)
[*] GOOGL: Return = 16.27% | Volatility = 1.11% -> Profile: Balanced / Moderate (Steady Positive Return, Low Volatility)
[*] AMZN : Return = 10.39% | Volatility = 1.38% -> Profile: Balanced / Moderate (Steady Positive Return, Low Volatility)
[*] MSFT : Return = -1.09% | Volatility = 0.80% -> Profile: Defensive / Review (Subdued Return or Higher Fluctuations)
```

Step 16: Educational Investment Analysis

```
=====
TASK 11: INVESTMENT RECOMMENDATIONS
=====
[DISCLAIMER]: Educational analysis only -- not financial advice.

[*] TSLA : Return = 57.95% | Volatility = 2.46% -> Profile: Aggressive Growth (High Return, Elevated Volatility)
[*] AAPL : Return = 24.51% | Volatility = 0.98% -> Profile: Balanced / Moderate (Steady Positive Return, Low Volatility)
[*] GOOGL: Return = 16.27% | Volatility = 1.11% -> Profile: Balanced / Moderate (Steady Positive Return, Low Volatility)
[*] AMZN : Return = 10.39% | Volatility = 1.38% -> Profile: Balanced / Moderate (Steady Positive Return, Low Volatility)
[*] MSFT : Return = -1.09% | Volatility = 0.80% -> Profile: Defensive / Review (Subdued Return or Higher Fluctuations)
```

Step 17: Generate Stock Price Visualization



Step 18: Generate Final Stock Summary Report

```

Practical-8 > stock_summary_report.txt
1 =====
2 | | | | FINEDGE SECURITIES LTD. - STOCK MARKET ANALYTICS REPORT
3 =====
4 1. DATASET DETAILS:
5   - Source File           : stock_data.csv
6   - Total Records         : 300
7   - Schema                : Timestamp (string), Stock (string), Price (double)
8   - Date Range Covered    : 2026-01-05 to 2026-03-05 (60 trading periods)
9
10 2. STOCKS ANALYSED:
11   - Evaluated Tickers     : AAPL, AMZN, GOOGL, MSFT, TSLA
12
13 3. STOCK PRICE SUMMARY (AVERAGE, MINIMUM, MAXIMUM):
14 Stock | Records | Avg Price ($) | Min Price ($) | Max Price ($)
15 -----
16 AAPL  | 60      | 202.22        | 181.07        | 230.34
17 AMZN  | 60      | 190.48        | 175.00        | 205.17
18 GOOGL | 60      | 178.72        | 159.69        | 191.89
19 MSFT  | 60      | 420.10        | 400.53        | 435.31

```

3. Key Questions / Analysis

Q1. Why is Apache Spark suitable for financial analytics?

Apache Spark is suitable for financial analytics because it can process large datasets using distributed and parallel computing. Spark DataFrames and built-in functions can efficiently perform aggregation, filtering and time-series analysis.

Q2. What is the purpose of grouping stock data by stock symbol?

Grouping stock data by stock symbol allows individual companies to be analysed separately.

Aggregation functions such as `avg()`, `min()` and `max()` can then be applied to calculate statistics for each stock.

For example:

```
df.groupBy("Stock").agg(avg("Price"))
```

calculates the average price for each stock.

Q3. How does average stock price help investors?

Average stock price provides a summary of the typical price level of a stock over the analysed period. It can be used along with other metrics such as returns and volatility to understand the historical behaviour of the dataset.

Q4. Why are maximum and minimum prices important?

Maximum and minimum prices show the price range observed during the analysed period. They help describe the highest and lowest recorded price levels and provide information about the historical price variation in the dataset.

Q5. How can Spark support real-time stock market monitoring?

Spark can process large volumes of streaming and historical data using distributed processing. Spark-based systems can be used to calculate metrics such as prices, returns and trends as new data becomes available.

4. Conclusion

The practical successfully demonstrated **Stock Market Analytics Using Window-Based Analysis** with PySpark.

A stock dataset containing **300 records across five stock symbols** was loaded into a Spark DataFrame. The schema and sample records were displayed, and aggregation functions were used to calculate average, minimum and maximum stock prices. Spark Window functions were then used to calculate **period returns, daily returns and moving averages**. Stock volatility was calculated using the standard deviation of daily returns. The analysis identified the calculated period return values for each stock and generated a stock summary report. The practical also demonstrated how PySpark can be used for time-series analysis and how Window functions such as lag() and rolling windows can perform calculations across ordered records.

A Matplotlib visualization of stock price trends was generated, and the complete analysis was saved in stock_summary_report.txt.

Note: The dataset used in this practical is a sample/generated dataset for educational purposes, so the calculated investment profiles should not be interpreted as real-world financial advice.

5. Github Link

Github Link: [24DIT041_BDA/Practical-8 at main · Krina-14/24DIT041_BDA](#)
